

PROYECTO FINAL DEL CURSO

Diseño e implementación de un compilador completo para un lenguaje de monitoreo de radares civiles

Objetivo	Aplicar de forma integral las fases del proceso de compilación mediante la construcción de un compilador académico completo, incluyendo análisis léxico, sintáctico y semántico, generación de código intermedio, traducción a código objeto y ejecución a través de una máquina virtual.
Valor	15 puntos

Descripción general del proyecto

Cada equipo desarrollará un compilador completo para un lenguaje de dominio específico denominado **RadarScript**, orientado al monitoreo de radares civiles y a la emisión de alertas meteorológicas y operativas en un contexto seguro y académico. El sistema deberá leer un programa fuente desde un archivo de texto, procesarlo en todas sus fases, generar archivos de salida intermedios y finales, y ejecutar el código objeto mediante una máquina virtual propia.

- El lenguaje deberá ser lo suficientemente pequeño para que el proyecto sea realizable durante el curso, pero lo bastante completo para evidenciar todas las etapas fundamentales de un compilador.
- El producto final deberá entregarse como una aplicación ejecutable que permita cargar un archivo fuente, compilarlo, visualizar resultados y ejecutar el programa compilado.
- El enfoque del proyecto es estrictamente académico: el lenguaje modelará operaciones de monitoreo, alertas, reportes y control de variables; no se utilizará para armamento ni para simulaciones ofensivas.

Especificaciones del lenguaje RadarScript

El lenguaje fuente deberá incluir, como mínimo, los siguientes elementos:

Categoría	Contenido mínimo
Tipos de datos	entero, decimal, cadena y booleano.
Instrucciones	declaración de variables, asignación, condicional si...entonces...fin, ciclo mientras...hacer...fin, alerta(), reporte() y comentarios.

Operadores	aritméticos (+, -, *, /), relacionales (>, <, ==, >=, <=, !=) y concatenación de cadenas.
Identificadores	deben iniciar con letra y pueden continuar con letras, números o guion bajo.
Archivo fuente	extensión sugerida .rdr.
Archivos de salida	como mínimo .lex, .sym, .int y .obj, además del reporte de errores cuando corresponda.

Ejemplo integral del flujo de compilación

A continuación se presenta un ejemplo completo del flujo esperado, desde el archivo fuente hasta la salida final de la máquina virtual. *Este ejemplo no es limitativo*, pero sí representa el nivel y el estilo de trabajo que debe observarse en el proyecto.

1. Archivo de entrada (.rdr)

```

programa monitoreo_clima;

entero distancia;
decimal viento;
cadena zona;
booleano alerta_activa;

distancia = 120;
viento = 85.5;
zona = "NORTE";
alerta_activa = verdadero;

si viento > 70 entonces
    alerta("Tormenta detectada en " + zona);
fin

mientras distancia > 0 hacer
    distancia = distancia - 30;
fin

reporte("Proceso finalizado");

```

Interpretación académica. El programa declara variables, realiza asignaciones, evalúa una condición, ejecuta un ciclo y emite un reporte. Por ello resulta adecuado para demostrar el recorrido completo del compilador.

2. Salida léxica (.lex)

```

PALABRA_RESERVADA    programa
IDENTIFICADOR         monitoreo_clima
PUNTO_Y_COMA          ;
TIPO                  entero
IDENTIFICADOR         distancia
PUNTO_Y_COMA          ;

```

TIPO	decimal
IDENTIFICADOR	viento
...	
PALABRA_RESERVADA	si
IDENTIFICADOR	viento
OPERADOR_RELACIONAL	>
NUMERO_DECIMAL	70
PALABRA_RESERVADA	entonces

La salida léxica deberá listar, como mínimo, el tipo de token y el lexema reconocido. Se valorará positivamente incluir línea y columna.

3. Tabla de símbolos (.sym)

distancia	: entero
viento	: decimal
zona	: cadena
alerta_activa	: booleano

4. Código intermedio (.int)

```
(=, 120, -, distancia)
(=, 85.5, -, viento)
(=, "NORTE", -, zona)
(=, verdadero, -, alerta_activa)
(>, viento, 70, t1)
(JF, t1, -, L1)
(CONCAT, "Tormenta detectada en ", zona, t2)
(ALERTA, t2, -, -)
(LABEL, -, -, L1)
(LABEL, -, -, L2)
(>, distancia, 0, t3)
(JF, t3, -, L3)
(-, distancia, 30, t4)
(=, t4, -, distancia)
(JMP, -, -, L2)
(LABEL, -, -, L3)
(REPORTE, "Proceso finalizado", -, -)
```

5. Código objeto (.obj)

```
MOV distancia, 120
MOV viento, 85.5
MOV zona, "NORTE"
MOV alerta_activa, verdadero

LOAD viento
PUSH 70
GT
JMPF L1

LOAD "Tormenta detectada en "
LOAD zona
CONCAT
ALERTA
```

```
L1:
L2:
LOAD distancia
PUSH 0
GT
JMPF L3

LOAD distancia
PUSH 30
SUB
STORE distancia
JMP L2

L3:
LOAD "Proceso finalizado"
REPORTE
HALT
```

6. Salida final esperada

```
ALERTA: Tormenta detectada en NORTE
REPORTE: Proceso finalizado
```

Fases obligatorias del compilador

- **Análisis léxico:** Lee el archivo fuente carácter por carácter, reconoce tokens y reporta errores léxicos tales como símbolos inválidos, cadenas mal cerradas o identificadores incorrectos.
- **Análisis sintáctico:** Valida que la secuencia de tokens cumpla con la gramática definida para RadarScript. Debe detectar estructuras inválidas y reportar el error con suficiente claridad.
- **Análisis semántico:** Verifica el significado del programa: variables declaradas antes de usarse, compatibilidad de tipos, duplicidad de identificadores y validez de expresiones y condiciones.
- **Generación de código intermedio:** Traduce las instrucciones del programa a una representación estructurada, por ejemplo cuádruplos, con temporales y etiquetas.
- **Generación de código objeto:** Convierte el código intermedio a un pseudoensamblador académico apto para ser interpretado por la máquina virtual.
- **Ejecución en máquina virtual:** Lee el archivo .obj, administra memoria y pila, interpreta instrucción por instrucción y produce la salida final del programa.

Máquina virtual: definición, funcionamiento e instrucciones

La máquina virtual es el componente que ejecuta el código objeto generado por el compilador. *En este proyecto no se exige ensamblador real ni código máquina del sistema operativo;* en su lugar, se emplea un pseudoensamblador académico que será interpretado por una VM propia. Esta decisión permite que el estudiante comprenda plenamente la fase final de la compilación sin volver el proyecto irrealizable.

Modelo de ejecución

- La VM recibe como entrada el archivo .obj generado por el compilador.

- Internamente administra, como mínimo, una memoria de variables, una pila de evaluación y un contador de programa.
- Lee una instrucción, la interpreta, actualiza el estado interno y continúa hasta encontrar HALT o hasta que ocurra un error de ejecución.

Explicación paso a paso de un fragmento

```
LOAD viento
PUSH 70
GT
JMPF L1
```

Secuencia de ejecución. *Primero se carga en la pila el valor de la variable **viento**. Luego se empuja el valor constante 70. La instrucción GT compara ambos elementos y deja en la pila un valor booleano. Finalmente, JMPF L1 revisa ese resultado: si es falso, transfiere el control a la etiqueta L1; si es verdadero, continúa con la siguiente instrucción.*

Instrucciones mínimas que debe soportar la máquina virtual

Instrucción	Descripción	Ejemplo de uso
MOV variable, valor	Asigna directamente un valor a una variable en memoria.	MOV distancia, 120
LOAD operando	Carga en la pila el valor de una variable o una cadena, según el diseño adoptado.	LOAD viento
PUSH valor	Introduce en la pila un valor constante literal.	PUSH 70
STORE variable	Toma el valor superior de la pila y lo guarda en la variable indicada.	STORE distancia
ADD	Extrae dos valores de la pila, los suma y apila el resultado.	LOAD a / LOAD b / ADD
SUB	Extrae dos valores de la pila, calcula la resta y apila el resultado.	LOAD distancia / PUSH 30 / SUB
MUL	Multiplica los dos valores superiores de la pila.	LOAD x / LOAD y / MUL
DIV	Divide los dos valores superiores de la pila.	LOAD total / LOAD n / DIV
GT	Compara si el primer operando es mayor que el segundo y devuelve verdadero o falso.	LOAD viento / PUSH 70 / GT

Instrucción	Descripción	Ejemplo de uso
LT	Compara si el primer operando es menor que el segundo.	LOAD x / PUSH 10 / LT
EQ	Determina si dos valores son iguales.	LOAD a / LOAD b / EQ
JMP etiqueta	Realiza un salto incondicional a la etiqueta indicada.	JMP L2
JMPF etiqueta	Realiza un salto si el valor booleano evaluado es falso.	JMPF L1
CONCAT	Concatena dos cadenas y deja el resultado en la pila.	LOAD "Alerta " / LOAD zona / CONCAT
ALERTA	Emite un mensaje de alerta a partir del valor disponible para salida.	ALERTA
REPORTE	Presenta un mensaje de reporte o conclusión del programa.	REPORTE
HALT	Finaliza la ejecución del programa.	HALT

Flujo completo del sistema

Archivo .rdr	Programa fuente escrito por el usuario en RadarScript.
Análisis léxico	Convierte caracteres en tokens.
Análisis sintáctico	Valida la estructura gramatical.
Análisis semántico	Verifica tipos, declaraciones y significado.
Código intermedio	Genera cuádruplos o una representación equivalente.
Código objeto	Traduce a pseudoensamblador académico.
Máquina virtual	Ejecuta el archivo .obj.
Salida	Muestra alertas, reportes o mensajes finales.

Especificaciones técnicas mínimas del proyecto

- El compilador deberá aceptar al menos un archivo fuente de entrada y generar los archivos de salida del proceso de compilación.
- El sistema deberá reportar errores con suficiente precisión, indicando al menos el tipo de error y la ubicación aproximada. Se valorará incluir línea y columna.
- El proyecto deberá evidenciar claramente la gramática utilizada y las reglas esenciales del lenguaje.
- La aplicación ejecutable deberá permitir, como mínimo, cargar un archivo, compilarlo, visualizar salidas intermedias y ejecutar el código objeto.
- Se permitirá el uso de Python, Java, C# u otro lenguaje de programación aprobado por el docente, siempre que el equipo justifique la elección y entregue el ejecutable correspondiente.

Entregables

Entregable	Descripción	Ponderación
Diseño del lenguaje	Definición de tokens, tipos de datos, instrucciones y gramática base.	2 puntos
Análisis léxico	Reconocimiento correcto de tokens y manejo de errores léxicos.	2 puntos
Análisis sintáctico	Validación de la gramática y reporte de errores sintácticos.	3 puntos
Análisis semántico	Tabla de símbolos y verificación de tipos y declaraciones.	2 puntos
Código intermedio	Generación correcta de cuádruplos o equivalente.	2 puntos
Código objeto y VM	Traducción a pseudoensamblador y ejecución funcional en la máquina virtual.	2 puntos
Ejecutable y documentación	Aplicación ejecutable, manual de usuario y manual técnico.	2 puntos

Observaciones finales

- El proyecto podrá desarrollarse en equipos de hasta tres estudiantes, salvo indicación distinta del docente.
- El énfasis de la evaluación estará en la comprensión integral de las fases del compilador y en la coherencia técnica entre el lenguaje diseñado, las salidas generadas y la ejecución final.

- No se espera un compilador industrial; se espera un compilador académico completo, funcional, bien estructurado y correctamente documentado.